MSC Project 2 (Conformance Predictions)

Introduction

The goal of this assignment is to design, implement, and evaluate a **conformance-aware predictive monitoring pipeline** that estimates the risk of future deviations for ongoing process instances. You will work with at least one real-life (or realistically simulated) event log and an explicit process model (e.g., Petri net, BPMN model converted to a behavioral representation). Starting from a classical conformance checking perspective, you will compute alignments or apply token-based replay to quantify how much each case deviates from the model and to identify the current "model state" of each running case. On top of these diagnostics, you will build supervised models that predict whether an ongoing case will eventually violate a compliance rule, incur a deviation above a certain cost threshold, or end in an undesired outcome attributable to non-conforming behavior. The assignment emphasizes the interaction between process models and data-driven learning: you must engineer conformance-related features, compare them against purely log-based features, and critically analyze the added value and computational cost of incorporating model-based information into predictive process monitoring.

Expected Deliverable (Implementation, Testing, and Prototype Software Documentation)

You are expected to deliver a **reproducible Python-based software artifact** (using PM4Py for process and conformance handling, scikit-learn for ML, and optionally a deep learning framework) together with tests and documentation.

The implementation must include:

1. A data and model ingestion component that:
   – loads and preprocesses the event log (filtering, timestamp and attribute cleaning, case selection), and
   – imports or discovers a process model and prepares it for conformance checking (e.g., Petri net with initial and final markings).

2. A conformance analysis module that computes, for each prefix of each case, alignment-based or token-based replay diagnostics, such as numbers and types of deviations (inserted/missing moves), cumulative alignment cost, model state/marking, and distance-to-completion indicators when feasible. Results must be cached or efficiently computed to handle repeated use during feature generation.

3. A prefix generation and feature engineering module that constructs:
   – standard log-based features (control-flow encodings, aggregated case attributes, temporal features), and
   – conformance-based features derived from the diagnostics (e.g., cumulative deviation cost, last deviation type, current model state, frequency of specific rule violations).

4. One or more supervised prediction pipelines for a clearly defined deviation-related target, for example: probability that the case's final alignment cost exceeds a threshold, probability of violating a specific constraint, or risk of ending in a known non-compliant end state. Pipelines should be implemented as scikit-learn–style objects (including preprocessing, feature selection, model training, and prediction).
5. Baseline models using only log-based features and enhanced models that also exploit conformance features. All key configurations (feature sets, algorithms, thresholds, hyperparameters) must be controlled via a small configuration module or file.
6. An end-to-end experiment runner (script or notebook) that starts from raw data and process model and reproduces the main results and plots without manual intermediate steps.

Software engineering requirements:
– Maintain the project in a Git repository with a meaningful commit history, making use of branches or issues for major changes.
– Organize the code into coherent modules (e.g., `io`, `conformance`, `features`, `models`, `evaluation`, `config`).
– Use docstrings and type hints where appropriate to document the purpose and interfaces of important classes and functions.
– Provide a `requirements.txt` or environment specification and a clear `README.md` explaining how to install dependencies and run the pipeline.

Testing:
You must implement automated tests (e.g., with `pytest`) for at least the following components:
– loading a small synthetic log and model;
– running conformance checking on a tiny example and verifying simple properties (e.g., zero deviations for a perfectly conforming trace);
– generating prefix-based features (including conformance features) and verifying shapes and basic consistency;
– training a simple model on a toy dataset and producing predictions without runtime errors.
Tests should be executable with a single command and provide a basic safety net against refactoring errors.

Prototype software documentation:
– A **technical documentation** (3–5 pages) describing the architecture, data/model flow, conformance checking strategy, feature design, and main implementation choices. An architecture diagram is strongly recommended.
– A **user guide** (in the `README` or separate) explaining how to: (1) place or obtain the log and model files; (2) configure the prediction task (target definition, thresholds, selected conformance features); and (3) run the full experiment pipeline.
– Optionally, a small **prototype interface** (e.g., Jupyter "dashboard" or CLI tool) that demonstrates online use: given the current prefix of a case, show its alignment diagnostics, predicted deviation risk, and key contributing features.

Expected Evaluation

Your evaluation must address the central question: **“Does conformance information derived from a process model improve the prediction of future deviations and compliance risks?”**

The experimental study should include:

1. A clear definition of the prediction problem and label construction: for example, defining a case as “deviant” if its final alignment cost exceeds a chosen threshold, if a particular rule is violated (e.g., an LTL constraint), or if conformance checking detects a specific type of non-compliant pattern. Motivations for the chosen definition must be documented.
2. A temporal evaluation protocol that avoids information leakage, such as splitting data into training/validation/test sets based on time or using rolling-origin evaluation. Cases must not be split across sets.
3. Training and tuning of at least one baseline model that uses no conformance-based features (only classical process mining encodings and attributes) and at least one conformance-aware model that augments these features with conformance diagnostics. Hyperparameter tuning should use a clearly described strategy (e.g., grid/random search with validation).
4. Use of appropriate evaluation metrics for the chosen target: for classification tasks, consider metrics such as ROC AUC, precision, recall, F1-score, precision–recall AUC, and calibration measures (Brier score, calibration curves); for regression-like deviation scores, use MAE, RMSE, or rank-based metrics depending on the goal. You are encouraged to analyze performance as a function of prefix length (early vs late warning).
5. Comparative analysis of baseline and conformance-aware models, including:
   – numerical comparisons (tables of metrics, confidence intervals or variability over multiple random seeds);
   – visualizations (ROC/PR curves, performance-by-prefix-length plots, feature importance analyses focusing on conformance-related variables);
   – qualitative inspection of example cases where conformance features significantly change the predicted risk.
6. Discussion of computational aspects: measure and report the runtime and resource requirements of conformance checking and feature generation, and reflect on the feasibility of deploying such a system in near real-time.
7. A critical discussion section in the report that reflects on:
   – the extent to which conformance features improved predictions in your scenario;
   – which specific conformance indicators were most useful and why;
   – potential threats to validity (quality of the process model, alignment noise, label definition);
   – limitations and ideas for future work (e.g., alternative model structures, approximate or incremental conformance, use of additional compliance languages).

The final report (approximately 10–15 pages excluding references and appendices) should connect your work to the broader literature on conformance checking and predictive process monitoring,

explaining both technical details and conceptual implications for organizations that wish to anticipate and mitigate non-compliant behavior.

References

You are encouraged to consult and cite the following (non-exhaustive) references and extend them with additional relevant work:

1. van der Aalst, W.M.P. (2016). *Process Mining: Data Science in Action* (2nd ed.). Springer.
2. Adriansyah, A., van Dongen, B.F., & van der Aalst, W.M.P. (2011). Conformance checking using cost-based fitness analysis. In *Enterprise, Business-Process and Information Systems Modeling* (BPMDS/EMMSAD).
3. Leemans, S.J.J., Fahland, D., & van der Aalst, W.M.P. (2013). Discovering block-structured process models from event logs containing infrequent behaviour. In *Business Process Management Workshops* (BPM).
4. Teinemaa, I., Dumas, M., La Rosa, M., & Maggi, F.M. (2019). Outcome-oriented predictive process monitoring: Review and benchmark. *ACM Transactions on Knowledge Discovery from Data*.
5. Verenich, I., Dumas, M., La Rosa, M., Maggi, F.M., & Teinemaa, I. (2019). Survey and cross-benchmark comparison of remaining time prediction methods in business process monitoring. *ACM Transactions on Intelligent Systems and Technology*.
6. Stevens, A., De Smedt, J., & Peeperkorn, J. (2021). Quantifying explainability in outcome-oriented predictive process monitoring. In *International Conference on Process Mining* (ICPM).

Additional literature on predictive compliance monitoring, constraint-based process modeling, and supervised process mining can be included as appropriate, provided it is properly cited in your report.